

Dokumentacja Techniczna Projektu: BacklogManager

1. Architektura i Działanie Systemu

1.1. Struktura Bazy Danych

Aplikacja wykorzystuje relacyjną bazę danych **SQLite**, zarządzaną przez system ORM **Entity Framework Core** w podejściu *Code-First*. Struktura opiera się na 5 głównych encjach oraz tabelach łączących, zapewniających pełną postać normalną i spójność (referential integrity) z wykorzystaniem kaskadowego usuwania danych (Cascade Delete).

- **Users:** Przechowuje dane autoryzacyjne (zahashowane algorytmem SHA256 hasła, e-mail, rola).
- **Games:** Globalny katalog gier. Przechowuje tytuł, rok wydania, dewelopera, gatunek oraz link do okładki (z wdrożonym mechanizmem *fallback* w przypadku wygasłych linków).
- **UserGames:** Tabela realizująca relację przypisania gry do biblioteki gracza. Przechowuje unikalne dla danego użytkownika informacje: status ukończenia gry oraz prywatne notatki.
- **Platforms & PlatformUserGame:** Zaimplementowano relację wiele-do-wielu (N:M). Gra w bibliotece użytkownika może być przypisana do wielu platform sprzętowych jednocześnie (np. PC, PS5).
- **Reviews:** Tabela agregująca oceny (w skali 1-10) i recenzje graczy. Posiada relacje 1:N z tabelą `Users` oraz `Games`, co pozwala na automatyczne wyliczanie i renderowanie średniej ocen w globalnym katalogu.

1.2. Główne Funkcje Systemu

Aplikacja realizuje pełen cykl CRUD (Create, Read, Update, Delete) w kilku wymiarach, oferując funkcjonalności z podziałem na role:

- **Zarządzanie Tożsamością (Auth):** Rejestracja, logowanie oparte na sesjach (Session State) oraz panel ustawień profilu z bezpieczną zmianą hasła.
- **Personalizowany Dashboard:** Strona główna dynamicznie generująca statystyki użytkownika oraz rekomendująca 4 losowe gry, których użytkownik jeszcze nie posiada (przy użyciu natywnej funkcji `EF.Functions.Random()`).
- **Katalog Gier i System Ocen:** Globalna lista gier z możliwością wyszukiwania, filtrowania po gatunkach oraz błyskawicznego dodawania do biblioteki (technologia AJAX / Fetch API). Katalog dynamicznie przelicza średnią ocen wystawionych przez społeczność.
- **Prywatna Biblioteka (Backlog):** Zarządzanie posiadanymi tytułami. Użytkownik może określać statusy (np. "W trakcie", "Ukończona"), przypisywać wielokrotne platformy, dodawać notatki oraz pisać i aktualizować recenzje gier ukończonych.
- **Panel Administratora:** Dedykowany moduł dla kont z rolą `Admin`. Pozwala na edycję danych gier w globalnym katalogu oraz zarządzanie użytkownikami (zmiana ról, kaskadowe usuwanie kont).

1.3. Sposób Działania Dockera

Projekt został w pełni skonteneryzowany z użyciem **Dockera**, co gwarantuje pełną przenośność (niezależność od środowiska hosta). Wdrożono proces *Multi-stage build* (wieloetapowe budowanie obrazu), opisany w pliku `Dockerfile`:

1. **Build (SDK)**: Obraz bazowy `mcr.microsoft.com/dotnet/sdk:8.0` pobiera zależności (NuGet), kompiluje kod źródłowy i wykonuje polecenie `dotnet publish`.
2. **Runtime**: Wygenerowane pliki przenoszone są do znacznie lżejszego obrazu bazowego `mcr.microsoft.com/dotnet/aspnet:8.0`, który służy wyłącznie do uruchomienia aplikacji.
3. Dzięki temu rozwiązaniu, kontener docelowy jest mały, bezpieczny i nie zawiera zbędnych narzędzi programistycznych. Aplikacja automatycznie nasłuchuje na porcie wewnętrznym 8080. Wbudowana klasa `DatabaseSeeder` dba o to, by przy każdym uruchomieniu w pustym środowisku, system wygenerował schemat bazy SQLite i wypełnił ją danymi początkowymi (konta, gry).

2. Uruchomienie i Wersjonowanie

2.1. Instrukcja Uruchomienia Projektu

Projekt można uruchomić na dowolnym systemie operacyjnym wyposażonym w aplikację *Docker Desktop*. Nie jest wymagane instalowanie środowiska .NET ani żadnych dodatkowych baz danych.

Krok 1: Pobranie repozytorium

`git clone https://github.com/Przemek738/BazyDanych_Projekt_Gry`

`cd BacklogManager`

Krok 2: Budowa obrazu Dockera Należy otworzyć terminal w głównym katalogu projektu (tam, gdzie znajduje się plik `Dockerfile`) i wywołać polecenie budujące obraz o nazwie `backlog-app`:

`docker build -t backlog-app .`

Krok 3: Uruchomienie kontenera Po zakończeniu budowy, uruchamiamy kontener, mapując port wewnętrzny (8080) na port lokalny (np. 8080):

`docker run -d -p 8080:8080 --name proj-backlog backlog-manager`

Krok 4: Używanie aplikacji Aplikacja jest gotowa do użycia. Należy otworzyć przeglądarkę internetową i wpisać adres: `http://localhost:8080`

Aby wyłączyć i usunąć aplikację po zakończonej pracy:

```
docker rm -f backlog-container
```

2.2. Wersjonowanie i Zarządzanie Kodem

Projekt jest wersjonowany przy użyciu systemu kontroli wersji **Git** i hostowany w zdalnym repozytorium **GitHub**.

- **Odizolowanie środowiska:** Plik `.gitignore` dba o to, by w repozytorium znajdował się wyłącznie kod źródłowy. Pliki konfiguracyjne IDE (`.idea/`, `.rider/`), pliki bazy danych (`*.db`) oraz wyniki kompilacji (`bin/`, `obj/`) nie są wysyłane na serwer, co zapewnia tzw. "czystość" repozytorium.
- Praktykuje się logiczne dzielenie pracy (częste commity z opisowymi wiadomościami), co ułatwia śledzenie zmian, cofanie błędów oraz ewentualną pracę zespołową nad kodem.

3. Schemat ERD:

